LAPORAN PRAKTIKUM

TEKNOLOGI IOT II

Dosen Pengampu:

1. Agus Ambarwati,S.Pd., M.Kom.
2. Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T., IPM., ASEAN Eng.



Disusun Oleh:

Machmud Al Hasan (NPM 25782044)

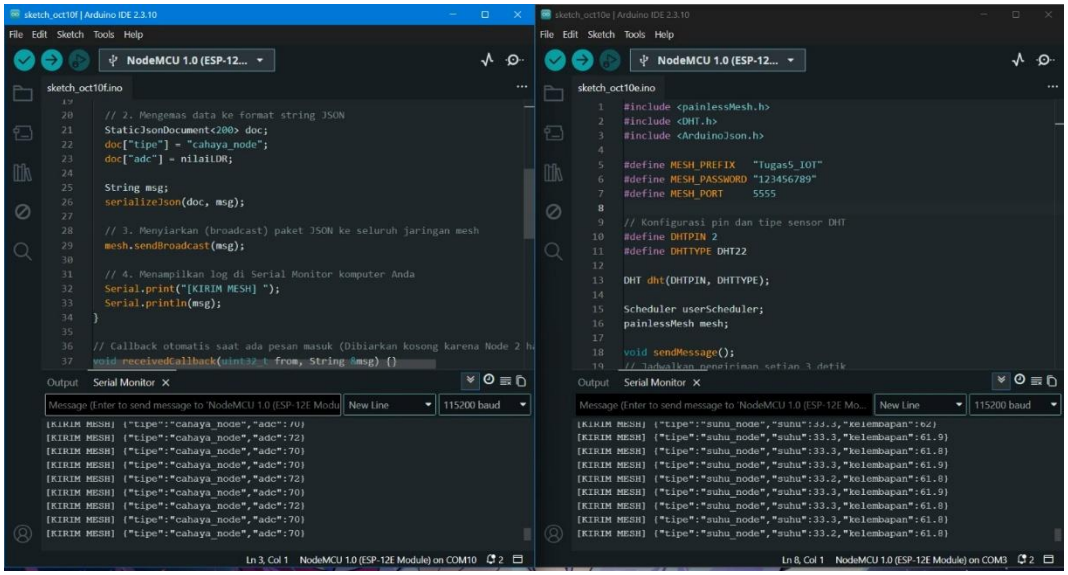
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

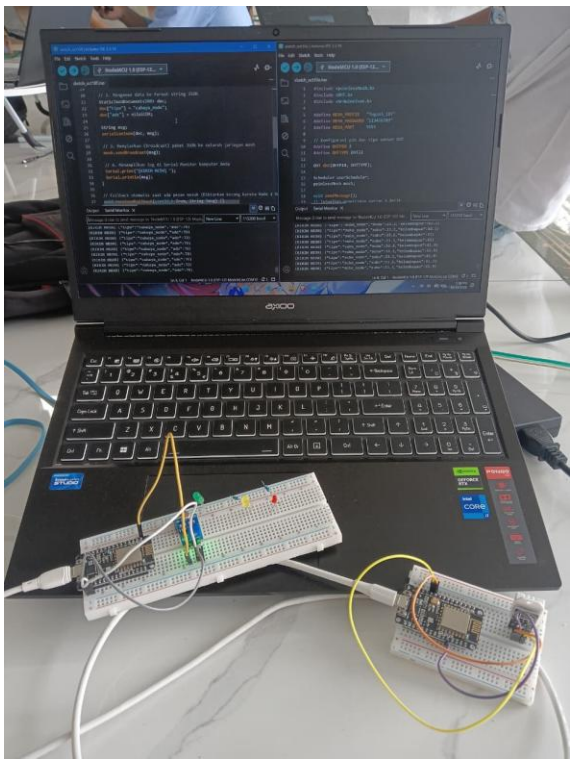
JURUSAN TEKNOLOGI INFORMASI

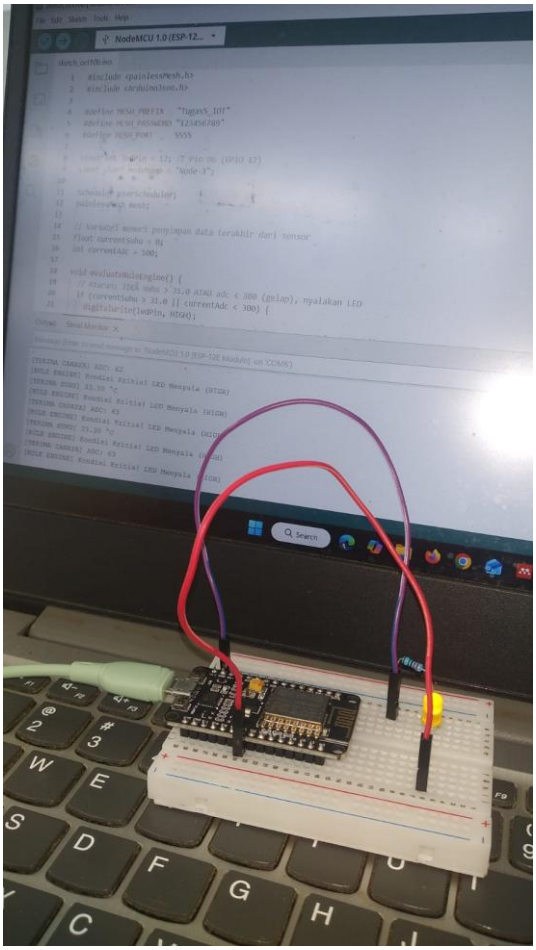
POLITEKNIK NEGERI LAMPUNG

TAHUN 2026

5.7 Tugas: Sistem Telemetri & Otomasi Terdistribusi Berbasis painlessMesh







Berdasarkan eksperimen dan dokumentasi pada Tugas 5.7, saya merancang sistem telemetri dan otomasi terdistribusi tanpa perantara router pusat (Access Point). Sistem ini memanfaatkan protokol painlessMesh untuk menghubungkan tiga buah NodeMCU: Node 1 sebagai sensor suhu, Node 2 sebagai sensor cahaya, dan Node 3 sebagai Hub aktuator. Berdasarkan foto dokumentasi praktikum, pengujian dilakukan dengan membagi layar laptop (split-screen) untuk memantau Serial Monitor dari beberapa NodeMCU secara bersamaan, guna memastikan transmisi mesh berjalan dua arah secara sinkron.

1. Perangkaian Perangkat Keras (Hardware)

Sesuai dengan dokumentasi foto, perangkaian dilakukan pada tiga board terpisah:

a. Node 1 (Stasiun Lingkungan): Modul sensor DHT dirangkai pada satu breadboard terpisah dan dihubungkan ke NodeMCU.
b. Node 2 (Stasiun Cahaya): Komponen LDR (ahaya resistor) dipasang pada breadboard lain sebagai pembaca intensitas ahaya ruang.
c. Node 3 (Hub Aktuator): Bertindak sebagai penerima, di mana saya merangkai sebuah LED berwarna kuning pada breadboard kecil beserta resistor pengamannya sebagai representasi aktuator cerdas.

2. Pemrograman Node 1 (Sensor Suhu & Kelembapan)

Berdasarkan tangkapan layar kode IDE di kanan atas, parameter jaringan mesh diatur menggunakan MESH_PREFIX "Tugas_IoT" dan MESH_PASSWORD "123456789". Node 1 berhasil mem- parsing pembacaan DHT menjadi format JSON dan mencetak log [KIRIM PESAN] {"tipe":"suhu_node", ...} ke Serial Monitor.

Dengan code lengkap sebagai berikut:

```
#include <painlessMesh.h>
#include <DHT.h>
#include <ArduinoJson.h>

#define MESH_PREFIX   "Tugas_IoT"
#define MESH_PASSWORD "123456789"
#define MESH_PORT     5555
#define DHTPIN 2
#define DHTTYPE DHT11 // Sesuaikan dengan DHT yang dipakai

DHT dht(DHTPIN, DHTTYPE);
Scheduler userScheduler;
```

```
painlessMesh mesh;

void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER,
&sendMessage);

void sendMessage() {
  float suhu = dht.readTemperature();
  float kelembapan = dht.readHumidity();

  if (isnan(suhu) || isnan(kelembapan)) return;

  StaticJsonDocument<200> doc;
  doc["tipe"] = "suhu_node";
  doc["suhu"] = suhu;
  doc["kelembapan"] = kelembapan;

  String msg;
  serializeJson(doc, msg);
  mesh.sendBroadcast(msg);

  Serial.print("[KIRIM PESAN] ");
  Serial.println(msg);
}

void setup() {
  Serial.begin(115200);
  dht.begin();
  mesh.setDebugMsgTypes(ERROR | STARTUP);
  mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler,
MESH_PORT);

  userScheduler.addTask(taskSendMessage);
  taskSendMessage.enable();
}

void loop() {
  mesh.update();
}
```

3. Pemrograman Node 2 (Sensor Cahaya LDR)

Sesuai tangkapan layar IDE di kiri atas, Node 2 mengemas data LDR ke dalam format JSON {"tipe":"cahaya_node", "adc": ...} dan berhasil melakukan broadcast secara berkala.

Dengan code lengkap sebagai berikut:

```
#include <painlessMesh.h>
#include <ArduinoJson.h>

#define MESH_PREFIX   "Tugas_IoT"
#define MESH_PASSWORD "123456789"
#define MESH_PORT     5555
#define LDR_PIN       A0

Scheduler userScheduler;
painlessMesh mesh;

void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER,
&sendMessage);

void sendMessage() {
  int nilaiADC = analogRead(LDR_PIN);

  StaticJsonDocument<200> doc;
  doc["tipe"] = "cahaya_node";
  doc["adc"] = nilaiADC;

  String msg;
  serializeJson(doc, msg);
  mesh.sendBroadcast(msg);

  Serial.print("[KIRIM PESAN] ");
  Serial.println(msg);
}

void setup() {
  Serial.begin(115200);
  mesh.setDebugMsgTypes(ERROR | STARTUP);
  mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler,
MESH_PORT);
```

```
  userScheduler.addTask(taskSendMessage);
  taskSendMessage.enable();
}

void loop() {
  mesh.update();
}
```

4. Pemrograman Node 3 (Hub Konsentrator & Rule Engine)

Node 3 tidak mengirimkan pesan, melainkan mengekstrak data dari JSON yang melayang di jaringan Mesh dengan MESH_PREFIX "Tugas_IoT". Jika suhu melebihi 31.0 Celcius ATAU sensor cahaya mendeteksi gelap (ADC di bawah 300), LED kuning fisik pada perangkat akan menyala.

Dengan code lengkap sebagai berikut:

```
#include <painlessMesh.h>
#include <ArduinoJson.h>

#define MESH_PREFIX   "Tugas_IoT"
#define MESH_PASSWORD "123456789"
#define MESH_PORT     5555
#define LED_PIN       12 // D6

Scheduler userScheduler;
painlessMesh mesh;

float lastSuhu = 25.0;
int lastAdc = 1024;

void receivedCallback(uint32_t from, String &msg) {
  StaticJsonDocument<200> doc;
  DeserializationError error = deserializeJson(doc, msg);
  if (error) return;

  String tipe = doc["tipe"].as<String>();

  // Sinkronisasi data masuk
  if (tipe == "suhu_node") {
    lastSuhu = doc["suhu"];
```

```
    Serial.printf("[UPDATE] Suhu dari %u: %.1f C\n", from, lastSuhu);
  } else if (tipe == "cahaya_node") {
    lastAdc = doc["adc"];
    Serial.printf("[UPDATE] Cahaya dari %u: %d\n", from, lastAdc);
  }

  // RULE ENGINE: Menyalakan LED Kuning
  if (lastSuhu > 31.0 || lastAdc < 300) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("-> AMBANG BATAS: LED Dinyalakan!");
  } else {
    digitalWrite(LED_PIN, LOW);
    Serial.println("-> NORMAL: LED Dimatikan.");
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  mesh.setDebugMsgTypes(ERROR | STARTUP);
  mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler,
MESH_PORT);

  mesh.onReceive(&receivedCallback);
}

void loop() {
  mesh.update();
}
```

5. Hasil Analisis Pengujian

Berdasarkan observasi di Serial Monitor (tampak pada laptop dan layar split), sistem berjalan sesuai instruksi. Node 1 dan Node 2 berhasil membungkus paket pembacaannya menjadi objek JSON secara independen, dan fungsi agregator di Node 3 sukses menangkap serta mengambil keputusan (trigger) terhadap lampu LED kuning, membuktikan jaringan ad-hoc beroperasi stabil. Eksperimen ini mengonfirmasi bahwa teknologi painlessMesh memungkinkan kita membangun smart system berskala lapangan tanpa harus menyediakan infrastruktur router konvensional.